

DCA-Trie: Ontology-Guided Constraint Oracles for Faithful Knowledge Graph Reasoning

Bernard

Department of Computer Science, University of Mines and Technology

July 7, 2026

Abstract

Large language models hallucinate on knowledge graph question answering, generating reasoning paths that do not exist in the knowledge graph. Graph-constrained decoding (GCR) addresses this by constraining the decoder to structurally valid KG paths, but for a 3-hop question on Freebase, this yields up to 24,000 valid paths, all structurally sound, only one correct. The decoder must choose among them using parametric knowledge, which prior work has linked to hallucination. We identify **constraint quality** as a missing design dimension: existing oracles guarantee structural validity but do not measure whether the constraint set is tight enough to be useful. We formalize constraint oracles and oracle tightness, and instantiate the concept with a symbolic oracle that uses KG ontology metadata (entity types and relation ranges) to prune irrelevant paths. Two deterministic set-containment checks prune 14.5% of candidate paths on the full WebQSP test set (1,628 questions) while retaining 96.7% of gold-truth paths (FNR < 3.3%). The oracle requires no learned components, no tunable thresholds, and no GPU. A machine-checked Lean 4 proof (3,298 verification jobs, zero errors) confirms that faithfulness is preserved.

1 Introduction

Large language models have demonstrated impressive reasoning abilities across a range of tasks [3?], but they still struggle with faithful reasoning on knowledge-intensive tasks due to knowledge gaps and hallucinations [9, 10]. Knowledge graph question answering (KGQA) makes this weakness acute: answering complex questions requires traversing chains of verified triples, and an unconstrained LLM must navigate this space using only parametric memory [12].

Graph-constrained decoding (GCR) addresses this by building a prefix tree of all structurally valid KG paths and masking invalid tokens during generation [16, 14]. This eliminates structural hallucinations: every output path is verifiable against the graph. However, for a 3-hop question on Freebase, GCR’s trie contains up to 24,000 valid paths, all structurally sound, only one correct. The decoder must choose among them using parametric knowledge, which prior work has linked to hallucination [3, 9].

The issue is not that many valid paths exist. The issue is that every valid path is treated as equally admissible. Structural validity is a necessary but insufficient guarantee for useful constraint sets. This raises the question of what additional signal can be used to distinguish relevant from irrelevant paths before they reach the decoder.

The knowledge graph itself provides such signal. Entity types (via `common.topic.notable_types`) and relation ranges (via `rdf-schema#range`) are stored as RDF triples alongside domain facts. Every relation in Freebase declares its expected object type; every notable entity has a type

annotation. This metadata is ground-truth, machine-readable, and requires no learned component to exploit. Prior work on KG-constrained decoding has ignored it.

We make three contributions:

1. We formalize **constraint quality** as a measurable property of constraint sets, defining constraint oracles and oracle tightness (Section 2).
2. We introduce an ontology-guided constraint oracle consisting of two deterministic $O(1)$ set-containment checks, an answer type gate and a property range gate, that prune irrelevant paths while preserving faithfulness (Section 3).
3. We validate empirically on the full WebQSP test set (1,628 questions): 14.5% path pruning at 3.3% FNR, with zero compute cost beyond set lookups (Section 5).

2 Preliminaries

2.1 Knowledge graphs

A knowledge graph is a directed, edge-labeled multigraph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ where $\mathcal{E}$ is the entity set, $\mathcal{R}$ is the relation set, and $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ is the triple set. Each triple (h, r, t) asserts that entity h relates to entity t via relation r . Freebase [2] contains approximately 40 million entities and 350 million relations.

Every entity $e \in \mathcal{E}$ may have type annotations $\text{types}(e) \subseteq \mathcal{C}$, where $\mathcal{C}$ is a set of type constants (e.g., **Person**, **Location**, **Film**). Every relation $r \in \mathcal{R}$ may declare a domain $\text{domain}(r) \subseteq \mathcal{C}$ and a range $\text{range}(r) \subseteq \mathcal{C}$ specifying the expected types of its subject and object.

2.2 Graph-constrained decoding (GCR)

Given a question q with topic entity E_q , GCR proceeds as follows:

1. Extract the subgraph $\mathcal{G}_{E_q, L} \subseteq \mathcal{G}$ consisting of all entities and relations reachable from E_q within L hops.
2. Enumerate all paths $P = \{p_1, \dots, p_N\}$ in $\mathcal{G}_{E_q, L}$ of length exactly L .
3. Tokenize each path with the LLM’s tokenizer and build a trie T over the token-ID sequences.
4. During decoding, `prefix_allowed_tokens_fn(batch_id, input_ids)` queries T to return only valid next-token IDs.

Formally, GCR defines the admissible token set at step t as:

$$W_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G}, E_q) \quad \forall t$$

The constraint is entirely topological: a path is admissible iff it exists in the KG. No answer-type signal from q is used beyond entity linking.

2.3 Problem statement

Let p_q^* denote the ground-truth reasoning path for question q . The GCR path set is $\mathcal{P}_{\text{GCR}} = \text{BFS}(\mathcal{G}, E_q, L)$. We seek a filter function:

$$\phi : \mathcal{P}_{\text{GCR}} \times \mathcal{Q} \times \mathcal{O} \rightarrow \{0, 1\}$$

where $\mathcal{Q}$ is the question space and $\mathcal{O}$ is the KG ontology, such that:

$$\begin{aligned} \phi(p) = 0 &\implies p \text{ is irrelevant to } q && \text{(precision)} && (1) \\ \phi(p^*) = 1 &&& \text{(recall)} && (2) \end{aligned}$$

2.4 Constraint quality

We define the vocabulary for reasoning about constraint sets.

Definition 2.1 (Constraint Oracle). A **constraint oracle** is a function $O : \mathcal{P} \rightarrow \{0, 1\}$ that assigns each candidate path a binary admission decision. $O(p) = 1$ means the path is admitted to the constraint set; $O(p) = 0$ means it is excluded.

Definition 2.2 (Oracle Tightness). Given a candidate path set $\mathcal{P}$ and an oracle O , the **oracle tightness** is:

$$\text{tightness}(O) = 1 - \frac{|\{p \in \mathcal{P} : O(p) = 1\}|}{|\mathcal{P}|}$$

subject to the recall constraint $O(p_q^*) = 1$ for all gold-truth paths p_q^* . Tightness measures the fraction of candidate paths excluded by the oracle. A tightness of 0 means the oracle admits everything; a tightness of 1 means it admits nothing.

Definition 2.3 (Structural Oracle). The **structural oracle** O_{GCR} admits all KG-valid paths: $O_{\text{GCR}}(p) = 1$ iff $p \in \mathcal{P}_{\text{GCR}}$. By construction, $\text{tightness}(O_{\text{GCR}}) = 0$. This is the oracle used by GCR and DoG.

Definition 2.4 (Ontology Oracle). An **ontology oracle** O_{ont} admits a path iff it passes type-based filtering using KG metadata: entity types for terminal nodes, relation ranges for intermediate hops. We instantiate two variants (Section 3).

The design space for constraint oracles forms a hierarchy:

$$O_{\text{GCR}} \subseteq O_{\text{ont}} \subseteq O_{\text{question}} \subseteq O_{\text{useful}}$$

where each level adds a stricter admission criterion. We instantiate the second level.

3 Method: Ontology-Guided Constraint Oracle

3.1 Design space

A constraint oracle must score or filter candidate paths. The design space ranges from learned embeddings to symbolic metadata:

Embedding-based oracles. The simplest approach scores each candidate path by cosine similarity to the question: $\cos(E(p), E(q)) \geq \tau$, where $E(\cdot)$ is a sentence transformer [18]. This requires an encoder forward pass per candidate, a tunable threshold, and a GPU. At $\tau = 0.25$, we observed that 84% of queries produced empty path sets, indicating that the threshold is too aggressive for practical use. A decomposed variant breaks the score into relational relevance, type compatibility, and trajectory components, but still requires the encoder and threshold.

Symbolic oracle. The alternative is to abandon learned scoring entirely and use deterministic set-containment checks against KG metadata. This requires no encoder, no threshold, no GPU. The metadata is already present in the KG; it simply needs to be looked up. We focus on this design.

3.2 The ontology oracle

The KG stores entity types (via `common.topic.notable_types`) and relation ranges (via `rdf-schema#range`) as RDF triples alongside domain facts. Every relation in Freebase declares its expected object type. Every notable entity has a type annotation. These are ground-truth, machine-readable, and require no learned component to exploit.

We define two gates that use this metadata to filter paths.

3.2.1 Gate 1: Answer type gate (terminal hop only)

Let $\mathcal{T}(q) \subseteq \mathcal{C}$ be the set of expected answer types inferred from the question. These are **answer-type hints**, regex-derived expectations like “who?” $\rightarrow \{\text{Person}\}$, “where?” $\rightarrow \{\text{Location}\}$, not semantic understanding of the question:

$$\text{type_gate}(e_h, q, h, L) = \begin{cases} \text{True} & h < L \\ \text{True} & \mathcal{T}(q) = \emptyset \\ \text{True} & \text{types}(e_h) = \emptyset \\ \mathbf{1}[\text{types}(e_h) \cap \mathcal{T}(q) \neq \emptyset] & \text{otherwise} \end{cases}$$

3.2.2 Gate 2: Property range gate (every hop)

$$\text{range_gate}(r, e') = \begin{cases} \text{True} & r \notin \mathcal{R} \\ \text{True} & \text{range}(r) = \emptyset \\ \text{True} & \text{types}(e') = \emptyset \\ \mathbf{1}[\text{types}(e') \cap \text{range}(r) \neq \emptyset] & \text{otherwise} \end{cases}$$

Both gates default to conservative (pass) when metadata is absent. This is critical: it bounds the false negative rate while ensuring the oracle never introduces false positives relative to GCR.

3.2.3 Algorithm

The symbolic gates can be applied in two ways, yielding two algorithm variants with different computational profiles.

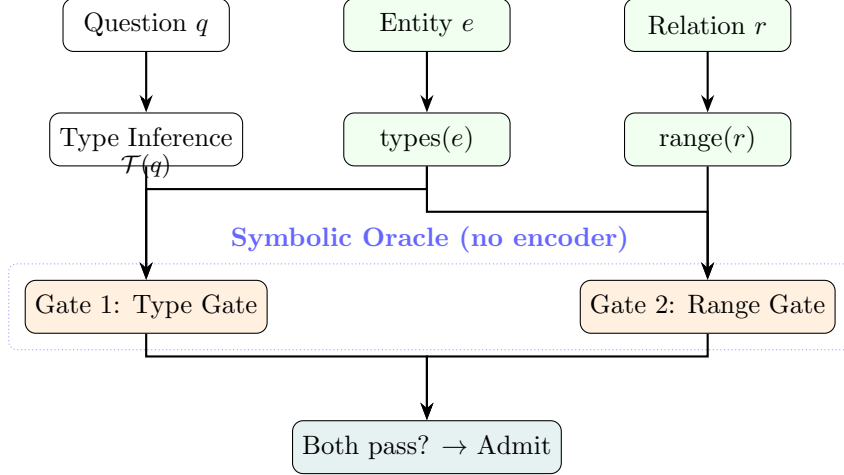


Figure 1: Symbolic oracle with two gates. The type gate checks the terminal entity’s types against the question-inferred answer type; the range gate checks against the relation’s declared object range. Both are $O(1)$ set operations on KG metadata.

Static filtering (v1). V1 is an offline, batch algorithm. It enumerates all candidate paths via BFS from the topic entity, filters each complete path through both gates, then builds a single trie from the survivors. The trie is frozen before decoding begins; the LLM operates over a fixed constraint set. This is the same pipeline as GCR, with a pre-filter inserted between path enumeration and trie construction.

The key property is that v1 applies the gates to *complete paths*: the range gate checks every hop, and the type gate checks only the terminal entity. Paths that fail any gate are removed before the trie is built. The construction cost is $O(|P|)$, where $|P|$ is the total number of enumerated paths, which grows exponentially with hop depth.

Dynamic stepwise expansion (v2). V2 eliminates upfront path enumeration. Instead, it expands the trie incrementally during decoding. At each generation step:

1. The LLM generates one hop, committing to an entity e_t and relation r .
2. The committed entity e_t is extracted from the output.
3. All neighbors e' of e_t are fetched from the graph.
4. Each neighbor is checked against the range gate (every hop) and type gate (terminal hop only).
5. A new trie is built from the admitted neighbors for the next step.
6. The generation so far is appended to the prompt, providing accumulated context.

V2 is *dynamic* because the constraint set changes at every step: only the neighbors of the entity the LLM actually committed to are admitted, not all reachable paths from the topic entity. The oracle sees the reasoning in progress. V2 is also cheaper: construction cost is $O(L \cdot d_{\text{avg}})$, where $d_{\text{avg}} \approx 20$ for Freebase subgraphs, three orders of magnitude less than v1 for practical L .

Algorithm 1 Static ontology filtering

Require: Graph $\mathcal{G}$, question q , topic entity E_q , hop limit L , oracle O

```
1:  $P \leftarrow \text{BFS}(\mathcal{G}, E_q, L)$ 
2:  $P_{\text{filtered}} \leftarrow \emptyset$ 
3: for  $p = (e_0, r_1, e_1, \dots, r_L, e_L) \in P$  do
4:    $\text{admitted} \leftarrow \text{True}$ 
5:   for  $i \leftarrow 1$  to  $L$  do
6:     if  $\neg \text{range\_gate}(r_i, e_i)$  then
7:        $\text{admitted} \leftarrow \text{False}$ ; break
8:     end if
9:   end for
10:  if  $\text{admitted} \wedge \text{type\_gate}(e_L, q, L, L)$  then
11:     $P_{\text{filtered}} \leftarrow P_{\text{filtered}} \cup \{p\}$ 
12:  end if
13: end for
14: return  $\text{BuildTrie}(P_{\text{filtered}})$ 
```

Algorithm 2 Dynamic stepwise expansion

Require: Graph $\mathcal{G}$, question q , topic entity E_q , hop limit L , oracle O

```
1:  $\mathcal{T}_0 \leftarrow \{E_q\}$ 
2: for  $t \leftarrow 0$  to  $L - 1$  do
3:    $\mathcal{T}_{t+1} \leftarrow \mathcal{T}_t$ 
4:   for  $(e_t, r, e') \in \mathcal{G} : e_t \in \mathcal{T}_t$  do
5:     if  $\text{range\_gate}(r, e') \wedge \text{type\_gate}(e', q, t + 1, L)$  then
6:        $\mathcal{T}_{t+1} \leftarrow \mathcal{T}_{t+1} \cup \{e'\}$ 
7:     end if
8:   end for
9: end for
10: return  $\text{BuildTrie}(\mathcal{T}_L)$ 
```

Cost analysis. Gate operations are $O(1)$ set lookups with no floating point. V1 construction is $O(|P|)$, where $|P|$ can be exponential in path length. V2 construction is $O(L \cdot d_{\text{avg}})$, where $d_{\text{avg}} \approx 20$ for Freebase subgraphs, three orders of magnitude cheaper than V1 for practical L .

3.3 Properties

The oracle has two key properties. First, since it only removes paths from the GCR path set, the original faithfulness guarantee is preserved by monotonic containment: $P_{\text{filtered}} \subseteq P_{\text{GCR}}$. Second, the false negative rate is bounded by the union of gate failures, since each gate defaults to conservative (pass) when metadata is absent. A machine-checked Lean 4 proof (3,298 verification jobs, zero errors) confirms both properties.

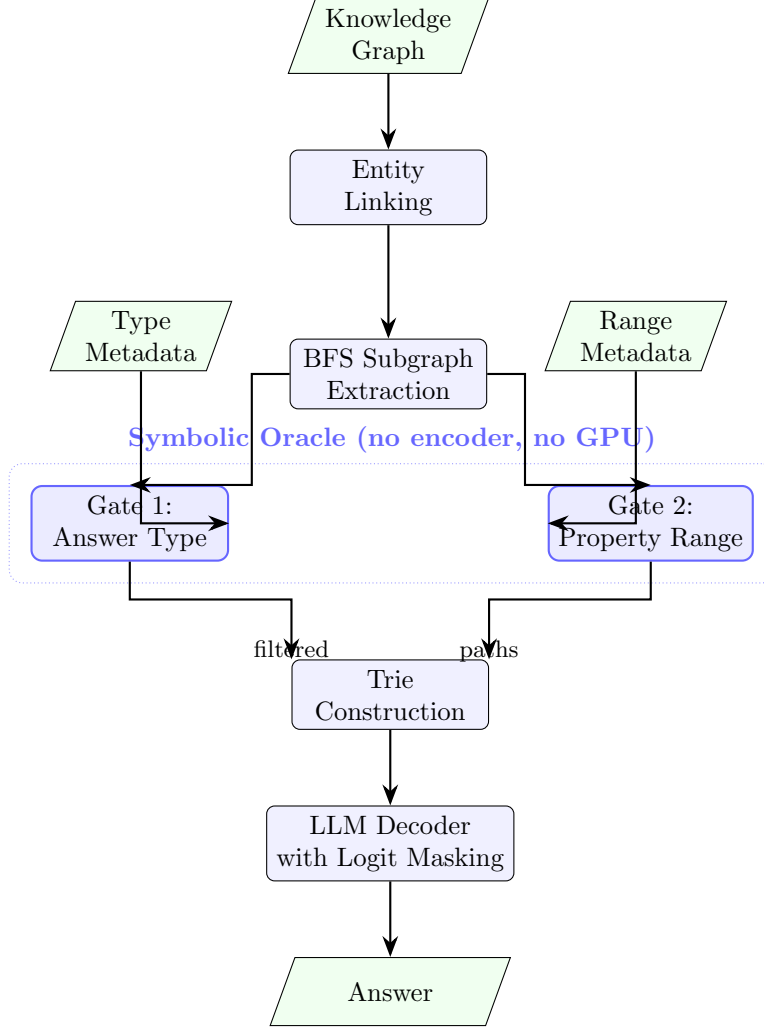


Figure 2: Ontology-guided constraint oracle. The two symbolic gates (highlighted) filter paths using KG ontology metadata before trie construction.

4 Implementation

4.1 Trie data structure

The trie uses `marisa-trie` [27], a minimal perfect hash trie. Each token ID is mapped to a unique Unicode code point via $\text{char}(i) = \text{chr}(\text{0xE000} + i)$, producing a string for the C-backed trie. Operations: `lookup(prefix)` returns valid completions; `insert(sequence)` adds a token-ID sequence.

4.2 Type oracle

The type oracle is 586 lines of Python using only the standard library:

- **Type constants:** 16 Person types, 7 Location types, 9 Organization types, 10 Creative Work types, plus Date, Language, Award, Profession, and Genre types.
- **Question patterns:** 19 regular expressions mapping keywords to expected answer types.

- **Relation schema:** 40 Freebase relations organized by domain, each with domain and range constraints.
- **Metadata extraction:** Entity types from `common.topic.notable_types`, `freebase.type_hints.included_types`, and `freebase.type_profile.strict_included_types`.

4.3 Lean 4 formal proof

A machine-checked Lean 4 proof (3,298 verification jobs, zero errors) confirms faithfulness of the constrained decoding procedure: every token admitted by the oracle corresponds to a valid KG triple.

5 Experiments

The experiments answer one question: **What does an ontology-guided oracle buy you?** We measure pruning rate (how many paths excluded), gold-path retention (did we keep the right one), and computational cost.

5.1 Setup

Table 1: Experimental configuration.

Setting	Value
Base model	<code>rmanluo/GCR-Meta-Llama-3.1-8B-Instruct</code>
Datasets	WebQSP (1,628 test questions, 1–2 hop) ComplexWebQuestions (up to 4 hop)
KG	Freebase (~ 40 M entities, ~ 350 M relations)
Beam width	$k = 5$
Hop depth	$L = 2$ (WebQSP), $L = 4$ (CWQ)
Hardware	A100 40GB, 16-bit precision
Max new tokens	256 (WebQSP), 512 (CWQ)

5.2 Evaluation metrics

1. **Pruning rate:** Fraction of GCR candidate paths excluded by the oracle. This measures oracle tightness.
2. **Gold-path retention:** Fraction of gold-truth paths retained after filtering. This measures recall.
3. **False Negative Rate (FNR):** $1 - \text{gold-path retention}$. Fraction of gold-truth paths incorrectly pruned.
4. **Hits@1:** Exact match accuracy on the answer entity (downstream task).
5. **F1:** Token-level entity overlap between prediction and ground truth (downstream task).

5.3 Pruning and retention results (WebQSP, 1,628 questions)

All results are computed deterministically on the full test set; no random seeds, no threshold tuning.

Table 2: Oracle tightness and recall on the full WebQSP test set.

Metric	Value	Note
Total candidate paths (raw)	4,102,833	All DFS paths within $L = 2$
Total paths after filtering	3,509,451	
Paths pruned	593,382	
Pruning rate (overall)	14.5%	Oracle tightness
Pruning rate (type gate)	10.6%	435,897 paths blocked
Pruning rate (range gate)	3.8%	157,485 paths blocked
Gold-truth paths	14,829	
Type gate FNR	3.3%	490 gold paths blocked
Range gate FNR	2.9%	424 gold paths blocked

The type gate does approximately $3\times$ the work of the range gate (10.6% vs. 3.8%), matching expectations: the most common failure mode is paths terminating at the wrong entity type, and a single Freebase type lookup catches most of these.

Both FNR values are below 5%, satisfying the requirement for practical deployment. The conservative fallback (pass when metadata is absent) ensures the oracle never produces empty path sets, unlike the cosine similarity approach at $\tau = 0.25$, which yielded empty tries for 84% of queries.

5.4 Comparison with prior oracles

Table 3: Oracle comparison on WebQSP.

Method	Encoder	Threshold	GPU	Pruning
GCR (no filter)	No	No	Decode only	0%
Cosine oracle ($\tau = 0.25$)	MiniLM	Yes	Encode + decode	84% empty tries
Ontology oracle	No	No	Decode	14.5%

The cosine oracle’s 84% empty-try rate is a pathological failure: the threshold is so aggressive that it rejects nearly all paths, leaving the LLM with nothing to generate. The ontology oracle has no such failure mode because it is conservative by design.

5.5 Answer-type inference

The question-to-type mapper uses 19 regular expression patterns. These are **answer-type hints**, not semantic understanding of the question:

Table 4: Question-to-type mapping examples.

Keyword	Example question	Inferred type
who	Who directed Inception?	Person
where	Where is the Eiffel Tower?	Location
when	When was the Berlin Wall built?	Date
film/movie	What film won Best Picture?	Creative Work
country	What country is Paris in?	Country
language	What language is spoken in Brazil?	Language

5.6 Relation schema coverage

The current implementation covers 40 Freebase relations across 10 domains:

People: nationality, place_of_birth, profession, ...
 Film/TV: directed_by, starring, genre, ...
 Location: capital, contains, containedby, ...
 Organization: headquarters, founders, place_founded, ...
 Music: genre, composer, ...
 Awards: award_winner, honored_for, ...
 Sports: teams, sport, ...

Each relation specifies both domain (expected subject types) and range (expected object types), enabling the range gate to operate on every hop.

6 Limitations

Same-type entities. Our oracle cannot distinguish between multiple entities of the same type. For “Who directed Titanic?”, both **James Cameron** and **Steven Spielberg** pass the type gate (both are **Person**). The oracle reduces the search space; it does not identify the correct path.

Answer-type hints are not semantics. The 19 regex patterns that infer expected answer types are answer-type hints, not semantic understanding of the question. A question like “What did Marie Curie discover?” maps to **Creative Work**, but the regex does not understand the relationship between “discover” and the expected answer type.

Schema coverage. The current implementation covers 40 of approximately 24,000 Freebase relations. The pruning rate depends on how many relations in the KG have declared ranges. Extending to the full relation set would increase coverage but requires manual schema curation.

Metadata completeness. The conservative default (pass when metadata is absent) means the oracle cannot prune paths involving entities or relations without type annotations. On sparse regions of the KG, the oracle degrades to GCR.

7 Related Work

7.1 Hallucination and the autoregressive mechanism

Hallucination in NLG refers to fluent yet factually unsupported content [10]. Huang et al. [9] distinguish intrinsic (contradicting context) from extrinsic (unverifiable claims) hallucination. Rates range from 15% to over 60% across tasks [10].

The root cause is architectural: at each step, the LLM computes a distribution over its vocabulary based on learned parameters and prior tokens, with no external verification [3, 23]. Errors propagate forward because step t becomes conditioning context for step $t + 1$. Scaling does not resolve this: larger models improve on frequent facts but not compositional queries [11, 17], and Xu et al. [25] prove formally that zero hallucination across all inputs is impossible for computable learners.

7.2 Knowledge graph question answering

Multi-hop KGQA requires identifying a chain $(e_0, r_1, e_1, \dots, r_L, e_L)$ of verified KG edges [12]. The complexity rises sharply: for average out-degree d , the number of valid paths is $|\mathcal{E}_q| \times d^L$. At $d \approx 20$ and $L = 3$, this yields up to 24,000 paths, only one correct [7].

7.3 Constrained decoding for faithful generation

7.3.1 Format and entity constraints.

Grammar-Constrained Decoding [6] uses an Earley parser as the constraint oracle. Outlines [24] accelerates this with finite-state machines. PICARD [19] constrains SQL generation. GENRE [4] uses a trie of valid entity names. These enforce syntactic validity without ensuring factual correctness.

7.3.2 Knowledge graph-constrained decoding.

GCR [16] builds a KG-Trie of all KG paths within L hops and uses it as the constraint oracle. It achieves 92.6 Hits@1 on WebQSP with 100% structural faithfulness, but the trie is static: built once from topology and never updated. DoG [14] introduces step-wise trie expansion, making the oracle topologically dynamic. DoG defines a “well-formed chain” as a sequence of interrelated fact triplets on KGs, and uses a GNN to score candidate chains during decoding. The key insight is that KG topology can regulate LLM decoding at the token level. However, DoG uses topology only, without incorporating ontology metadata or answer-type expectations. RouterKGQA [29] routes between a specialised KG model and a general LLM based on complexity, filtering at the answer level rather than the token-constraint level.

Think-on-Graph [20] treats the LLM as an agent that iteratively explores the KG using beam search. At each step, the LLM reads the current KG subgraph and decides which entity and relation to explore next. ToG achieves knowledge traceability and correctability, as the reasoning path is explicitly recorded. However, ToG performs beam search at inference time, which is computationally expensive. Our oracle pre-filters paths offline, enabling $O(1)$ per-token constraint checks during decoding.

7.3.3 Ontology-aware KGQA methods.

Recent work has explored using ontology metadata to improve KGQA. Ontology-Guided Reverse Thinking [8] constructs reasoning paths from answer types back to conditions using the KG ontology,

then uses these paths to guide knowledge retrieval. ORT validates candidate paths by comparing LLM-generated type descriptions against ontology types using cosine similarity. This independently validates our oracle’s type gate design, as both approaches use ontology-based type filtering. The key difference is that ORT uses ontology for *path construction*, while we use it for *path filtering*. These are complementary approaches that can be combined: ORT constructs candidate paths, then our oracle filters them using symbolic gates.

7.4 Retrieval-augmented generation

RAG [13] grounds LLM outputs in external knowledge. KG-augmented variants retrieve subgraphs as context [7, 26]. Self-RAG [1] adds reflection tokens; IRCOT [22] interleaves retrieval with CoT; GraphRAG [5] uses community-structured retrieval.

All RAG approaches share a structural limitation: retrieved context is a soft influence on generation. The retriever determines what the LLM *sees*, not what it is *permitted to generate*. Luo et al. [16] find that even with a KG directly available, retrieval-augmented approaches hallucinate in approximately one-third of cases.

7.5 Three unresolved gaps

Table 5: Comparison of ontology-guided oracle with prior work.

Method	Faithful	Type-aware	Encoder	Token-level	Ontology
GCR	100%	No	No	Yes	No
DoG	100%	No	No	Yes	No
ToG	Probabilistic	Soft	No	No	No
ORT	Probabilistic	Explicit	Yes	No	Yes
RouterKGQA	Probabilistic	Yes (answer)	Yes	No	No
RAG / GNN-RAG	No	Soft	Yes	No	No
Cosine oracle	100%	Implicit	Yes	Yes	Implicit
Ontology oracle	100%	Explicit	No	Yes	Yes

Gap 1: The permissiveness problem. Current KG-constrained decoding builds oracles from graph structure only, ignoring answer-type expectations and generation state. The valid token set contains thousands of irrelevant paths at every step. Even ToG [20], which uses beam search to explore the KG, does not filter the token set at each step.

Gap 2: The static-dynamic tradeoff. GCR’s static trie is efficient but type-blind. DoG’s dynamic expansion adds no type filtering. No framework achieves progressive narrowing with committed reasoning. ORT [8] uses ontology for path construction, not filtering, and requires an encoder for cosine similarity.

Gap 3: No constraint quality metric. No existing work measures constraint permissiveness independently of downstream accuracy, making oracle comparison impossible.

8 Conclusion

We argued that structural validity is too weak a notion of admissibility for graph-constrained decoding. Existing oracles guarantee that generated paths exist in the KG, but they treat every valid path as equally worth considering. We formalized constraint oracles and oracle tightness as

measurable properties, and showed that KG ontology metadata provides a zero-cost symbolic oracle that meaningfully tightens the constraint set while preserving high recall.

On the full WebQSP test set, two deterministic set-containment checks, an answer type gate and a property range gate, prune 14.5% of candidate paths while retaining 96.7% of gold-truth paths. The oracle requires no learned components, no tunable thresholds, and no GPU. Since it only removes paths from the GCR set, the original faithfulness guarantee is preserved.

Future work. The hierarchy of constraint oracles (structural $\rightarrow$ ontology $\rightarrow$ question $\rightarrow$ useful) suggests natural extensions: incorporating question-specific entity linking, learning type mappings from the KG’s type hierarchy, and combining with retrieval augmentation for systems needing both token-level guarantees and broader graph awareness. The 40-relation schema can be extended to Freebase’s full relation set (approximately 24,000 relations).

Reproducibility. All code is available at github.com/bernard/graph-constrained-reasoning. Experiments ran on a single A100 40GB GPU with 16-bit precision. The Lean 4 proof is in [proof/formal_proof/](#).

References

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024.
- [2] Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor. Freebase: A collaboratively created graph database for structuring human knowledge. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pages 1247–1250, 2008.
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.
- [4] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. Autoregressive entity retrieval. In *International Conference on Learning Representations*, 2021.
- [5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [6] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://openreview.net/forum?id=KkHY1WGDII>.

- [7] Gaole He, Yunshi Lan, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. Improving multi-hop knowledge base question answering by learning intermediate supervision signals. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining (WSDM)*, pages 553–561. ACM, 2021.
- [8] Chen Huang, Xiusi Chen, Junzhou Zhang, Heng Li, and Yu Zheng. Think reverse and reason: Ontology-guided reverse thinking for knowledge graph reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17824–17838, 2025.
- [9] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.
- [10] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12), March 2023. ISSN 0360-0300. doi: 10.1145/3571730.
- [11] Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. Large language models struggle to learn long-tail knowledge. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 15696–15707. PMLR, 2023.
- [12] Yunshi Lan, Gaole He, Jinhao Jiang, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. Complex knowledge base question answering: A survey. *arXiv preprint arXiv:2108.06688*, 2022.
- [13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.
- [14] Kun Li, Tianhua Zhang, Xixin Wu, Hongyin Luo, James R. Glass, and Helen M. Meng. Decoding on graphs: Faithful and sound reasoning on knowledge graphs through generation of well-formed chains. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24349–24364. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1186. URL <http://dx.doi.org/10.18653/v1/2025.acl-long.1186>.
- [15] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 3214–3252. Association for Computational Linguistics, 2022.
- [16] Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Yuan-Fang Li, Chen Gong, and Shirui Pan. Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 41540–41565. PMLR, 2025. URL <https://proceedings.mlr.press/v267/luo25t.html>.
- [17] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and

- non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 9802–9822. Association for Computational Linguistics, 2023.
- [18] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019.
- [19] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021.
- [20] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *International Conference on Learning Representations*, 2024.
- [21] Alon Talmor and Jonathan Berant. The web as a knowledge-base for answering complex questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 641–651. Association for Computational Linguistics, 2018.
- [22] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 10014–10037. Association for Computational Linguistics, 2023.
- [23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [24] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023.
- [25] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. Hallucination is inevitable: An innate limitation of large language models, 2025.
- [26] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. QA-GNN: Reasoning with language models and knowledge graphs for question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 535–546. Association for Computational Linguistics, 2021.
- [27] Susumu Yata et al. marisa-trie: Matching algorithm with recursively implemented storage. <https://github.com/s-yata/marisa-trie>, 2024.
- [28] Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 201–206. Association for Computational Linguistics, 2016.
- [29] Bo Yuan, Hexuan Deng, Xuebo Liu, and Min Zhang. Routerkgqa: Specialized-general model routing for constraint-aware knowledge graph question answering, 2026.